BTS SIO · OPTION SISR · ÉPREUVE E6

Installation et configuration de Snipe-IT sur Ubuntu 24.04

Gestion des actifs informatiques (ITAM) — Apache · PHP · MariaDB

Procédure complète, commandes commentées et captures de la maquette

Auteur	Jack Kakou — étudiant BTS SIO option SISR, IEF2I (Vincennes)
Date	23 octobre 2026
Version du document	1.0
Logiciel	Snipe-IT 8.x (branche stable du dépôt officiel) — solution libre de gestion des actifs (ITAM)
Système	Ubuntu 24.04 LTS « Noble » — machine virtuelle VMware

Résumé. Ce document décrit, étape par étape, le déploiement de Snipe-IT sur un serveur Ubuntu 24.04 : installation de la pile Apache / PHP / MariaDB, création d'une base dédiée, récupération du code avec Git, installation des dépendances avec Composer, configuration et sécurisation, publication par un VirtualHost, puis prise en main sur un cas concret (création d'un actif et affectation à un utilisateur). Chaque étape précise **à quoi elle sert** et ce que fait chaque commande.

Sommaire

N°	Partie
1	Objectifs et contexte
2	Architecture et prérequis
3	Procédure d'installation (étapes 1 à 13)
4	Vérifications finales
5	Fonctionnement de Snipe-IT
6	Exemple d'utilisation : cycle de vie d'un actif
7	Analyse critique
8	Dépannage : problèmes courants et solutions
9	Bilan et suite du projet
10	Sources

1. Objectifs et contexte

Snipe-IT est une solution libre (open source) de **gestion des actifs informatiques** (IT Asset Management, ITAM), développée avec le framework PHP Laravel. Elle centralise l'inventaire, le suivi et la gestion du cycle de vie des ressources d'une organisation : ordinateurs, licences, accessoires, consommables et composants. Son interface web donne une visibilité en temps réel sur l'ensemble du parc.

L'objectif de cette réalisation est de déployer un serveur Snipe-IT fonctionnel sur une machine virtuelle Ubuntu 24.04, accessible par un nom de domaine interne (**http://snipeit.local**), puis de le valider par un cas d'usage réel : enregistrer un matériel et l'affecter à un collaborateur.

Les cinq piliers de Snipe-IT

Pilier	Rôle
Actifs (Assets)	biens immobilisés identifiés par une étiquette d'inventaire (Asset Tag), avec suivi de leur état
Licences	actifs immatériels : conformité aux contrats et attribution des licences logicielles
Accessoires et consommables	suivi des stocks de périphériques et de fournitures distribués aux utilisateurs
Composants	éléments internes (mémoire, disques…) destinés à la mise à niveau ou à la réparation des actifs
Utilisateurs (People)	association des ressources aux personnes, pour la responsabilité et la traçabilité

Compétences visées (E6)

- installer et configurer un service sur un serveur Linux (Apache, PHP, MariaDB) ;
- sécuriser un déploiement : utilisateur de base de données dédié, droits de fichiers, clé de chiffrement ;
- documenter une procédure technique et valider le résultat par des tests.

2. Architecture et prérequis

Environnement de travail

Élément	Valeur
Machine	machine virtuelle VMware
Système d’exploitation	Ubuntu 24.04 LTS « Noble »
Serveur web	Apache 2.4.58 (paquet 2.4.58-1ubuntu8.10)
Langage	PHP 8.3 (version fournie par Ubuntu 24.04) avec les extensions requises
Base de données	MariaDB 10.11 (version fournie par Ubuntu 24.04)
Gestionnaire de dépendances	Composer 2.9.5
Application	Snipe-IT, installé dans `/var/www/snipe-it`
Nom d’accès	`snipeit.local` (nom interne, résolu par le fichier hosts)

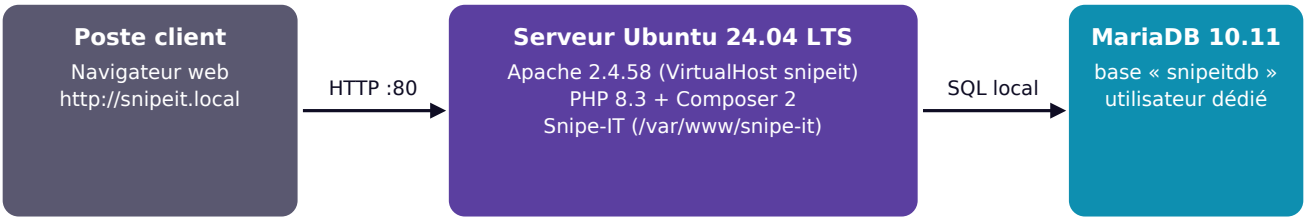
Prérequis de Snipe-IT

Les paquets fournis par Ubuntu 24.04 suffisent : aucun dépôt externe n’est nécessaire.

Composant	Exigence Snipe-IT	Ubuntu 24.04	Conforme
PHP	8.2 ou supérieur	8.3	Oui
Base de données	MySQL 5.7+ ou MariaDB	MariaDB 10.11	Oui
Serveur web	Apache avec mod_rewrite (ou Nginx + PHP-FPM)	Apache 2.4.58	Oui
Extensions PHP	curl, mbstring, xml, zip, gd, bcmath, mysql, fileinfo, sodium, exif	installées à l’étape 2	Oui
Extension PHP facultative	ldap (uniquement pour un annuaire)	installée à l’étape 2	Oui

Architecture mise en place

Les briques sont installées sur le même serveur : Apache reçoit les requêtes HTTP des postes, PHP exécute le code Laravel de Snipe-IT, et MariaDB stocke les données (actifs, utilisateurs, historique).



3. Procédure d’installation

Les commandes sont lancées depuis un terminal, avec un compte membre du groupe *sudo*. Les valeurs entre chevrons (par exemple `<MotDePasseSolide>`) sont à remplacer par vos propres valeurs.

1 Mettre à jour le système

À quoi ça sert ? Partir d’un système à jour garantit que les paquets installés ensuite sont cohérents entre eux et corrigés des failles connues.

```
sudo apt update && sudo apt upgrade -y
```

Commande	Rôle
`apt update`	actualise la liste des paquets disponibles
`apt upgrade -y`	installe les mises à jour sans demander de confirmation

2 Installer Apache, PHP et les outils nécessaires

À quoi ça sert ? Snipe-IT est une application web PHP : Apache la sert aux navigateurs et PHP exécute son code. Les extensions PHP sont installées en une seule fois pour que toutes les dépendances soient satisfaites dès le premier démarrage. Git et Composer serviront à récupérer le code et ses bibliothèques.

```
sudo apt install -y apache2 libapache2-mod-php php php-cli php-mysql php-curl php-gd \
     php-mbstring php-xml php-zip php-bcmath php-intl php-ldap git unzip curl
sudo systemctl enable --now apache2
```

Paquet	Rôle
`apache2, libapache2-mod-php`	serveur web et module qui exécute PHP dans Apache
`php-mysql`	connexion de PHP à MariaDB
`php-curl, php-gd`	requêtes HTTP ; traitement des images (logos, photos d’actifs)
`php-mbstring, php-xml, php-zip, php-bcmath`	caractères accentués, lecture XML, archives, calculs précis
`php-ldap`	facultatif : liaison à un annuaire (Active Directory)
`git, unzip, curl`	récupération du code source et de Composer

Sur la maquette, Apache était déjà présent : apt indique que la version 2.4.58 est la plus récente.

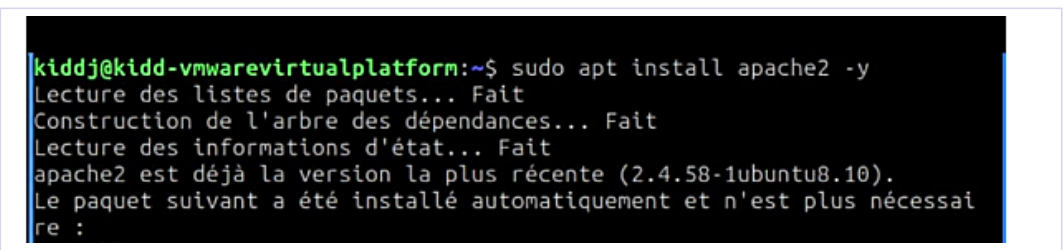


Figure 1 — Installation d’Apache : le paquet apache2 est déjà à la dernière version disponible (2.4.58).

3 Installer et sécuriser MariaDB

À quoi ça sert ? MariaDB stocke toutes les données de Snipe-IT. Une installation neuve contient des éléments prévus pour les tests (comptes anonymes, base « test »). Le script de sécurisation les supprime et protège le compte root.

```
sudo apt install -y mariadb-server
sudo mariadb-secure-installation
```

On répond « Oui » aux questions de nettoyage (utilisateurs anonymes, accès root distant, base de test) et on définit une authentification root adaptée.

4 Créer la base de données et l'utilisateur dédié

À quoi ça sert ? Snipe-IT ne doit jamais utiliser le compte administrateur de MariaDB. On crée une base réservée et un utilisateur qui n'a de droits que sur elle (principe du moindre privilège) : en cas de compromission de l'application, les autres bases du serveur restent protégées.

```
sudo mariadb
CREATE DATABASE snipeitdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'snipeit_user'@'localhost' IDENTIFIED BY '<MotDePasseSolide>';
GRANT ALL PRIVILEGES ON snipeitdb.* TO 'snipeit_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Instruction	Rôle
`CREATE DATABASE snipeitdb …`	crée la base ; utf8mb4 gère tous les caractères (accents, symboles)
`CREATE USER … @'localhost'`	crée l'utilisateur, autorisé à se connecter uniquement depuis le serveur
`GRANT ALL PRIVILEGES ON snipeitdb.*`	donne les droits sur cette seule base
`FLUSH PRIVILEGES`	recharge les droits en mémoire

Mot de passe. Choisir un mot de passe long et unique, jamais réutilisé, et ne pas le laisser visible sur une capture d'écran ou dans le document rendu.

5 Récupérer le code source de Snipe-IT avec Git

À quoi ça sert ? Le code est cloné depuis le dépôt officiel dans le dossier des sites web. Avec Git, les mises à jour futures s'obtiennent par une simple commande (git pull) plutôt qu'en retéléchargeant une archive.

```
cd /var/www/
sudo git clone https://github.com/grokability/snipe-it snipe-it
cd snipe-it
```

Le dépôt s'appelait auparavant *snipe/snipe-it* (nom visible sur la capture) ; GitHub redirige l'ancienne adresse vers le dépôt actuel. Le clonage récupère environ 250 Mio d'historique.

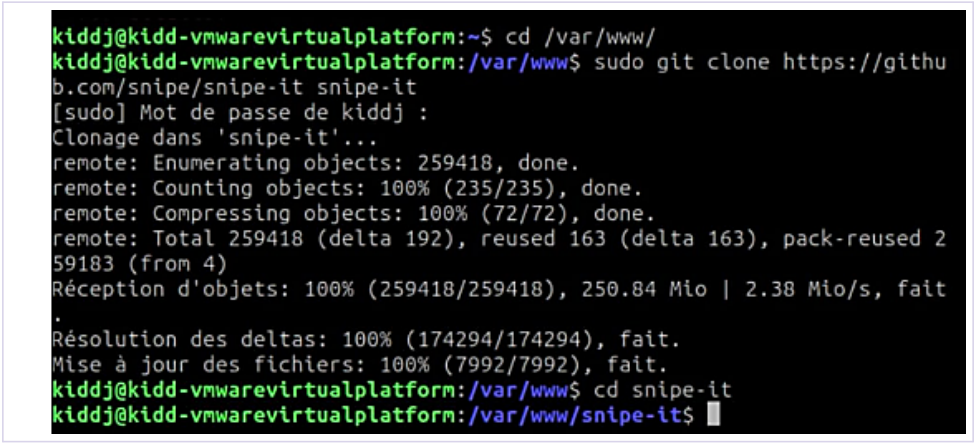


Figure 2 — Clonage du dépôt Snipe-IT dans /var/www puis entrée dans le dossier snipe-it.

6 Installer Composer

À quoi ça sert ? Composer est le gestionnaire de dépendances de PHP : il télécharge les bibliothèques dont dépend Laravel. Son installation globale permet de l'appeler depuis n'importe quel dossier et simplifie les mises à jour.

```
cd /tmp
curl -sS https://getcomposer.org/installer | php
sudo mv /tmp/composer.phar /usr/local/bin/composer
composer --version
```

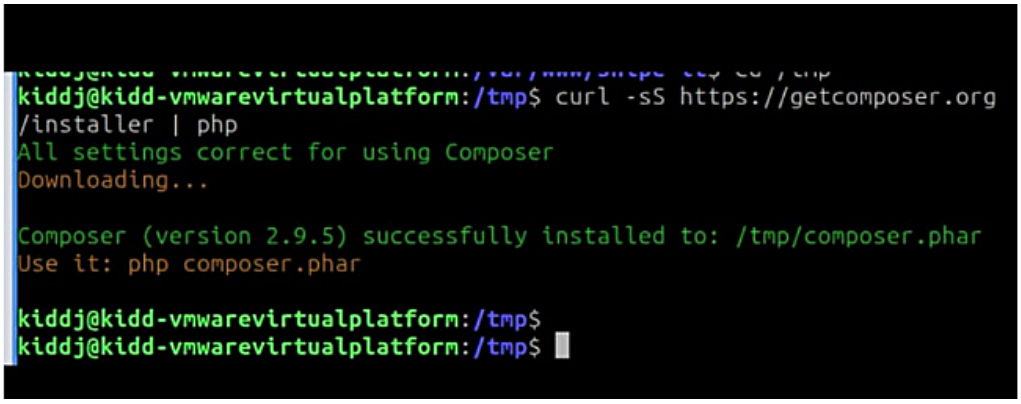


Figure 3 — Téléchargement de l'installateur : Composer 2.9.5 installé dans /tmp/composer.phar.

Le déplacement vers `/usr/local/bin` rend la commande *composer* disponible pour tous les utilisateurs.

7 Installer les dépendances de l’application

À quoi ça sert ? Cette étape télécharge toutes les bibliothèques nécessaires au framework Laravel, aux versions exactement prévues par les développeurs de Snipe-IT. L’option --no-dev exclut les outils de développement, inutiles en production.

```
cd /var/www/snipe-it
sudo composer install --no-dev --prefer-source
```

Composer avertit qu’il ne faut pas l’exécuter en *root* : le script pourrait lancer du code avec tous les droits. Sur la maquette de test, la confirmation est demandée (réponse « yes »). En production, on préfère lancer la commande avec le compte propriétaire du dossier (par exemple `sudo -u www-data`).

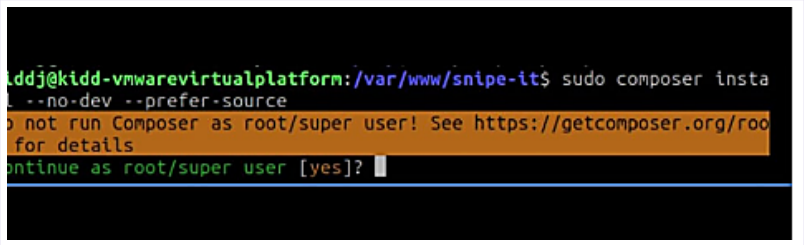


Figure 4 — Composer demande confirmation avant de continuer en tant que root/super utilisateur.

8 Créer et renseigner le fichier de configuration .env

À quoi ça sert ? Le fichier .env contient les paramètres propres à l’installation : adresse du site, fuseau horaire et accès à la base de données. On part du modèle fourni (.env.example) pour ne rien oublier.

```
sudo cp .env.example .env
sudo nano .env
```



Figure 5 — Copie du modèle .env.example vers .env.

Dans le fichier, on renseigne notamment les paramètres suivants :

```
APP_ENV=production
APP_DEBUG=false
APP_URL=http://snipeit.local
APP_TIMEZONE=Europe/Paris
DB_HOST=localhost
DB_DATABASE=snipeitdb
DB_USERNAME=snipeit_user
DB_PASSWORD=<MotDePasseSolide>
```

Paramètre	Rôle
APP_ENV, APP_DEBUG	mode production et débogage désactivé (aucun détail technique affiché aux utilisateurs)
APP_URL	adresse du site, utilisée pour les liens et notifications
APP_TIMEZONE	fuseau horaire : les dates des historiques doivent être correctes
DB_*	informations de connexion à la base créée à l'étape 4

9 Générer la clé de chiffrement de l'application

À quoi ça sert ? Laravel chiffre les sessions et les données sensibles avec une clé unique à chaque installation. Sans elle, l'application refuse de démarrer ; la perdre rendrait certaines données illisibles : elle doit être conservée avec les sauvegardes.

```
sudo php artisan key:generate
```

Comme l'application est en mode production, Artisan demande une confirmation (« Are you sure you want to run this command? ») : on répond Yes.

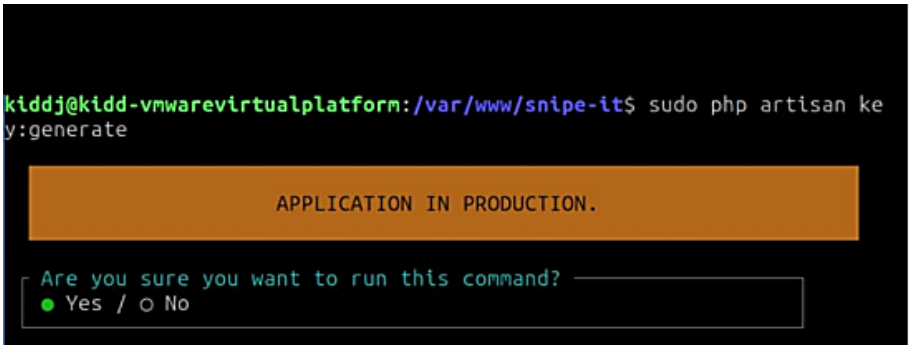


Figure 6 — Génération de la clé d'application : confirmation demandée en mode production.

10 Appliquer les droits d'accès

À quoi ça sert ? Le serveur web s'exécute avec l'utilisateur www-data. Il doit pouvoir écrire dans les dossiers de stockage (journaux, fichiers téléversés) mais pas modifier le reste du code : un paramétrage rigoureux évite l'altération ou la suppression accidentelle de fichiers système.

```
sudo chown -R www-data:www-data /var/www/snipe-it
sudo chmod -R 755 /var/www/snipe-it/storage
sudo chmod -R 755 /var/www/snipe-it/public/uploads
```

Commande	Rôle
chown -R www-data:www-data	donne la propriété du dossier à l'utilisateur du serveur web
chmod -R 755 …/storage	lecture et parcours pour tous, écriture pour le propriétaire seulement
chmod -R 755 …/public/uploads	même règle pour les fichiers envoyés par les utilisateurs



Figure 7 — Attribution du dossier à www-data et réglage des permissions (la capture applique 755 à l'ensemble du dossier).

11 Publier le site : VirtualHost Apache et nom snipeit.local

À quoi ça sert ? Un VirtualHost dédié isole le site dans sa propre configuration et le rend accessible par un nom lisible. Le DocumentRoot pointe sur le dossier *public* de Laravel : le code source et le fichier .env restent hors de portée du navigateur.

```
sudo nano /etc/apache2/sites-available/snipeit.conf
```

```
<VirtualHost *:80>
    ServerName snipeit.local
    DocumentRoot /var/www/snipe-it/public

    <Directory /var/www/snipe-it/public>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/snipeit_error.log
    CustomLog ${APACHE_LOG_DIR}/snipeit_access.log combined
</VirtualHost>
```

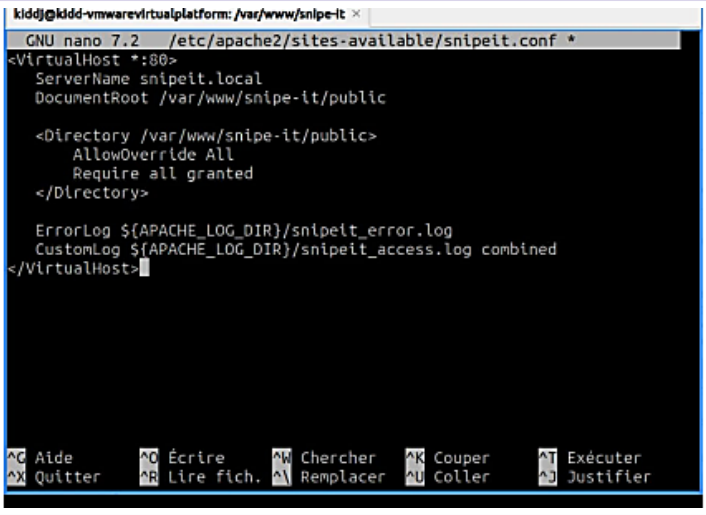


Figure 8 — Fichier /etc/apache2/sites-available/snipeit.conf édité avec nano.

On active ensuite le module de réécriture d'URL (indispensable à Laravel), le nouveau site, puis on recharge Apache :

```
sudo a2enmod rewrite
sudo a2dissite 000-default.conf
sudo a2ensite snipeit.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

Commande	Rôle
`a2enmod rewrite`	active la réécriture d'URL (sur la maquette : déjà activé)
`a2dissite 000-default.conf`	désactive la page d'accueil par défaut d'Apache
`a2ensite snipeit.conf`	active le site Snipe-IT
`apache2ctl configtest`	vérifie la syntaxe : doit répondre « Syntax OK »
`systemctl reload apache2`	applique la configuration sans couper le service

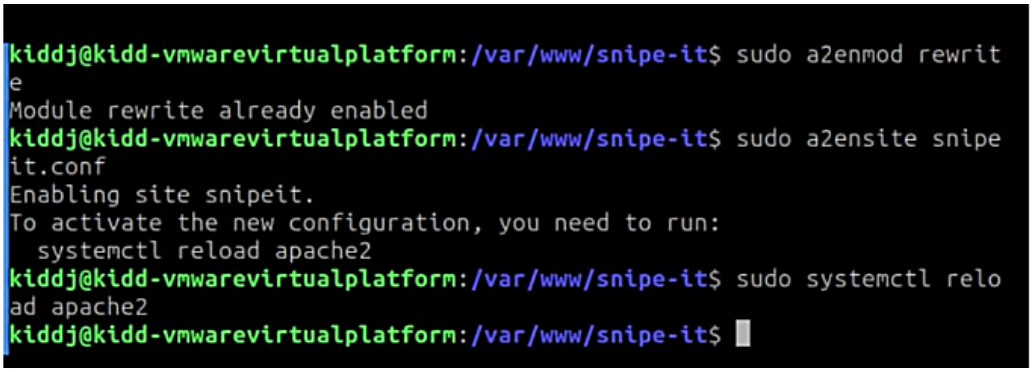


Figure 9 — Activation du module rewrite et du site snipeit.conf, puis rechargement d'Apache.

Le nom *snipeit.local* n'existe pas dans un DNS public : on l'associe à l'adresse du serveur dans le fichier hosts de chaque poste client (`/etc/hosts` sous Linux, `C:\Windows\System32\drivers\etc\hosts` sous Windows) :

```
<IP-du-serveur>   snipeit.local
```

12 Terminer l'installation avec l'assistant web et le test de pré-installation

> **À quoi ça sert ?** Au premier accès, Snipe-IT lance un diagnostic qui vérifie que l'environnement est conforme (URL, base de données, droits, mode production…). Une fois tous les contrôles au vert, l'assistant crée les tables de la base puis le compte administrateur.

Depuis un navigateur, on ouvre **http://snipeit.local/setup**. Chaque ligne du diagnostic doit afficher une coche verte : base de données connectée, fichier .env non exposé, application en mode production, fichiers appartenant à www-data, dossier de stockage inscriptible, mode debug désactivé, bibliothèque d'images installée.

Les étapes suivantes de l'assistant permettent de créer les tables de la base, puis le compte administrateur (nom, adresse e-mail, identifiant, mot de passe fort). Les paramètres de messagerie peuvent être testés avec le bouton « Send Test » si un serveur SMTP est disponible.

13 Se connecter à Snipe-IT

> **À quoi ça sert ?** La page de connexion et le tableau de bord confirment que l'application, la base de données et le serveur web fonctionnent ensemble. Le tableau de bord est le point de départ de toute l'administration.

Après connexion, le tableau de bord affiche des compteurs (actifs, licences, accessoires, consommables, composants, utilisateurs). Ils sont à zéro tant que rien n'est enregistré : l'environnement est prêt pour l'importation ou la création des premiers actifs.

4. Vérifications finales

Contrôle	Commande ou action	Résultat attendu
Services démarrés	systemctl is-active apache2 mariadb	« active » deux fois
Démarrage automatique	systemctl is-enabled apache2 mariadb	« enabled » deux fois
Configuration Apache	sudo apache2ctl configtest	Syntax OK
Connexion à la base	mariadb -u snipeit_user -p snipeitdb -e "SELECT 1"	la requête aboutit
Accès web	http://snipeit.local depuis un poste	page de connexion Snipe-IT
Pré-test	http://snipeit.local/setup	tous les contrôles en vert
Fichier .env protégé	http://snipeit.local/.env	introuvable ou interdit (404 / 403)
Mode production	APP_ENV=production, APP_DEBUG=false	aucune erreur détaillée affichée

5. Fonctionnement de Snipe-IT

Snipe-IT repose sur une structure simple qui garantit la traçabilité de l'inventaire : il distingue les **actifs** (matériel doté d'un numéro de série et d'une étiquette uniques) des accessoires, consommables et composants.

5.1 La structure de navigation

Un menu latéral donne accès aux catégories d'inventaire et à la gestion des utilisateurs. Chaque catégorie de ressources est isolée, ce qui permet une gestion fine des stocks.

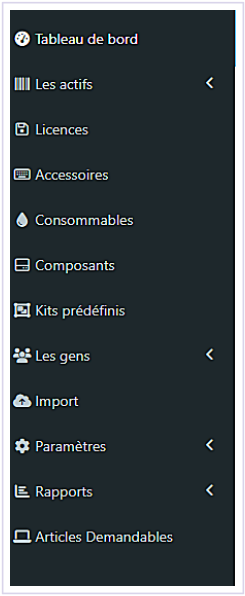


Figure 10 — Menu latéral : tableau de bord, actifs, licences, accessoires, consommables, composants, kits, utilisateurs, import, paramètres, rapports.

Menu	Contenu
Les actifs	matériel inventorié, avec statut et utilisateur affecté
Licences	licences logicielles et nombre de postes autorisés
Accessoires / Consommables	périphériques distribués ; fournitures à stock limité
Composants	pièces internes (mémoire, disques) rattachées aux actifs
Kits prédéfinis	ensembles d'éléments attribués ensemble (poste de travail type)
Les gens	utilisateurs et groupes
Import	import de fichiers CSV pour alimenter l'inventaire
Paramètres, Rapports	configuration générale ; exports et rapports d'activité

5.2 Création rapide

Le bouton « Create New » de la barre supérieure centralise les actions d'ajout : on crée un actif, une licence, un accessoire, un consommable, un composant ou un utilisateur sans quitter la page courante.

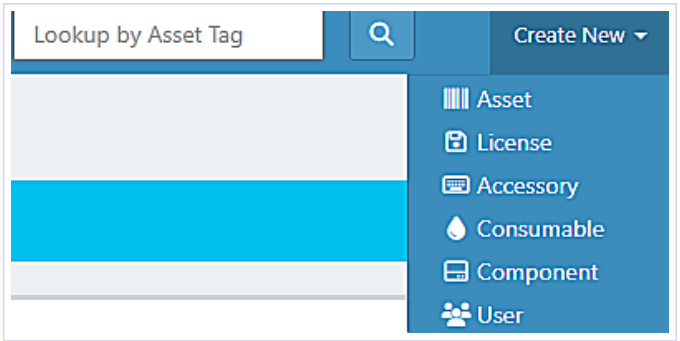


Figure 11 — Menu « Create New » : création rapide d'un actif, d'une licence, d'un accessoire, d'un consommable, d'un composant ou d'un utilisateur.

6. Exemple d'utilisation : cycle de vie d'un actif

Cet exemple suit un ordinateur de bureau, de sa création dans l'inventaire à son affectation à un collaborateur, puis à la lecture de l'historique. Les données (utilisateur, adresse e-mail) sont fictives.

Étape 1 — Créer l’actif

On renseigne l’étiquette d’inventaire (Asset Tag) unique, le numéro de série, le **modèle** et le **statut** initial. Lier chaque matériel à un modèle prédéfini garantit l’intégrité des données et partage les caractéristiques techniques communes. Le statut « Ready to Deploy » indique que l’actif peut être affecté.

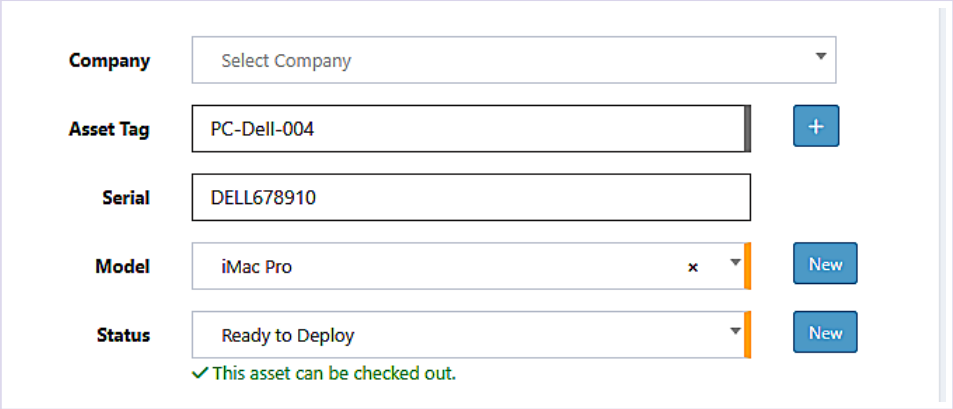


Figure 12 — Formulaire de création d’un actif : Asset Tag PC-Dell-004, numéro de série, modèle et statut « Ready to Deploy ».

Étape 2 — Créer l’utilisateur

Pour affecter l’actif, la personne doit exister dans la base. On saisit son prénom, son nom, son nom d’utilisateur, un mot de passe et son adresse e-mail. La case « Cet utilisateur peut se connecter » reste décochée si la personne n’a besoin d’aucun accès à l’application.



Figure 13 — Création d’un utilisateur de test (onglet Information).

Étape 3 — Affecter l’actif (Checkout)

L’action **Checkout** lie le matériel à l’utilisateur. Le système confirme l’opération par un bandeau vert et met à jour la fiche : statut « Deployed », utilisateur assigné, date de remise.

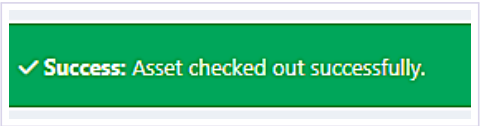


Figure 14 — Confirmation : « Asset checked out successfully ».

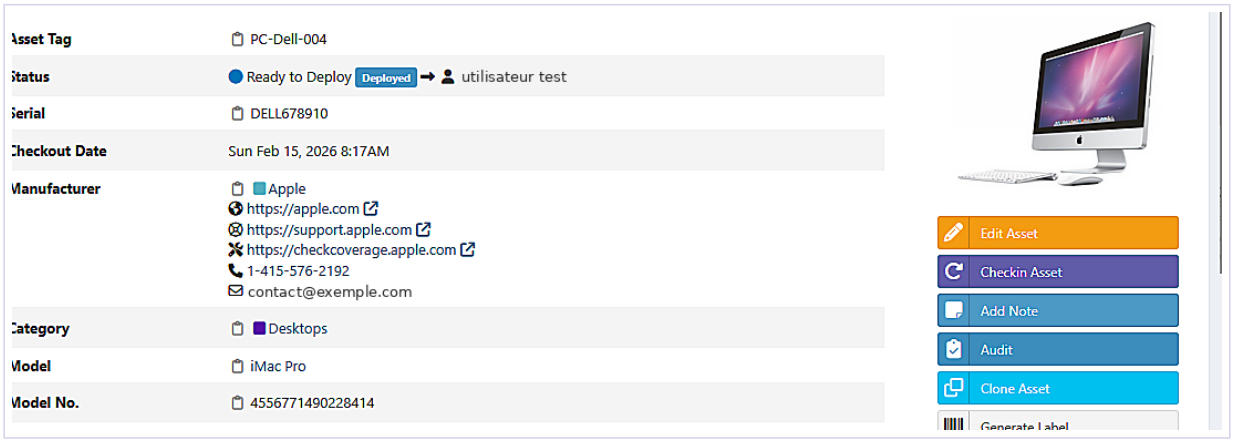


Figure 15 — Fiche détaillée de l'actif après affectation : statut déployé, numéro de série, date de checkout, fabricant, catégorie, modèle.

Étape 4 — Consulter l'historique

Toutes les actions (création, modification, affectation) sont enregistrées chronologiquement : c'est la piste d'audit du matériel. Ici, l'actif est créé à 8 h 15 puis affecté à 8 h 17, avec la note « BUREAU RH ».

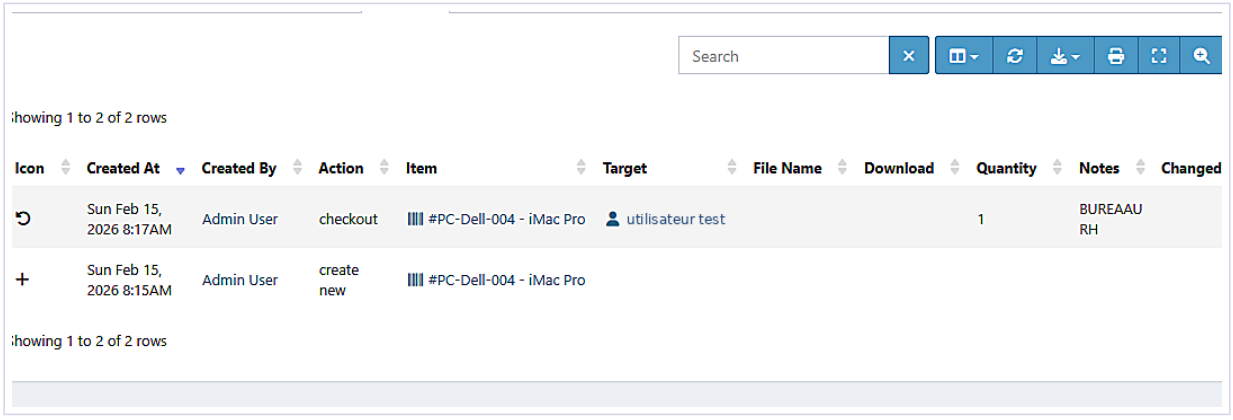


Figure 16 — Journal d'activité : création de l'actif puis checkout vers l'utilisateur.

7. Analyse critique

Points forts

- **Gratuité** : solution libre, sans licence à acheter ;
- **installation maîtrisable** : paquets d'Ubuntu 24.04 et dépôt officiel suffisent ;
- **traçabilité** : chaque création et chaque affectation est historisée (qui, quand, quoi) ;
- **interface claire** : menu modulaire, création rapide, tableau de bord lisible ;
- **évolutivité** : import CSV, extension LDAP, rapports et API.

Limites et améliorations

Limite constatée	Amélioration proposée
Accès en HTTP : identifiants en clair sur le réseau	activer HTTPS (certificat interne ou public) et rediriger le port 80 vers 443
Nom snipeit.local résolu par fichiers hosts, poste par poste	créer un enregistrement dans le DNS interne
Comptes gérés localement	lier Snipe-IT à l'annuaire LDAP / Active Directory (extension php-ldap prête)
Composer lancé en root sur la maquette	exécuter composer avec l'utilisateur www-data

Limite constatée	Amélioration proposée
Pas de sauvegarde automatisée	planifier une sauvegarde de la base (mariadb-dump) et des dossiers storage, public/uploads et du fichier .env
Inventaire alimenté manuellement	utiliser l'import CSV pour les volumes importants, et la tâche planifiée pour les alertes

8. Dépannage : problèmes courants et solutions

Symptôme	Cause probable	Solution
Avertissement « Do not run Composer as root »	commande lancée avec sudo	lancer avec l'utilisateur www-data, ou confirmer sur une maquette
Erreur 500 ou page blanche	droits insuffisants sur storage ; clé absente	refaire chown / chmod ; vérifier key:generate
Erreur 404 sur toutes les pages sauf l'accueil	module rewrite inactif ou AllowOverride absent	a2enmod rewrite ; AllowOverride All ; reload
Page « It works » d'Apache	site par défaut encore actif	a2dissite 000-default.conf puis a2ensite snipeit.conf
Pré-test : base de données en erreur	identifiants .env faux ou utilisateur sans droits	vérifier DB_* et les GRANT
snipeit.local introuvable	nom absent du fichier hosts du poste	ajouter la ligne IP + nom dans hosts
Modifications du .env sans effet	cache de configuration	php artisan config:clear

9. Bilan et suite du projet

Snipe-IT est déployé sur Ubuntu 24.04 : la pile Apache / PHP / MariaDB est installée, une base dédiée est prévue avec un utilisateur à droits limités, le code et ses dépendances sont installés avec Git et Composer, et le site est publié par un VirtualHost. Le cycle de vie d'un actif (création, affectation, historique) est présenté en partie 6.

Compétences mobilisées : installation et configuration d'un service sous Linux, administration d'une base de données, gestion des droits, utilisation de Git et de Composer, lecture de documentation technique et validation par des tests.

Suites prévues :

- mise en place du HTTPS et d'un nom DNS interne ;
- liaison à l'annuaire LDAP / Active Directory ;
- import du parc existant (fichier CSV) et définition des catégories, modèles et statuts ;
- sauvegarde régulière de la base et des fichiers, et planification des tâches (alertes) avec cron.

10. Sources

- Snipe-IT, documentation officielle (prérequis et installation) — snipe-it.readme.io
- Snipe-IT, page de téléchargement et versions — snipeitapp.com/download
- Dépôt officiel du projet — github.com/grokability/snipe-it

Les captures de terminal proviennent de la maquette VMware du projet. Les captures de la partie 6 illustrent le cycle de vie d'un actif avec des données de démonstration ; les noms et adresses y ont été remplacés par des données fictives.